

Time Series Analysis

Havana Rika



Ben-Gurion University
of the Negev

Decomposing a time series

Seasonal decomposition is the **process by which we deconstruct a time series into its components** typically, **trend**, **seasonality**, and **residuals**.

The general approach for decomposing a time series is as follows:

1. **Detrending**: Here, we estimate the trend component (which is the smooth change in the time series) and remove it from the time series, giving us a detrended time series.
2. **Deseasonalizing**: Here, we estimate the seasonality component from the detrended time series. After **removing** the **seasonal** component, what is left is the **residual**.

Detrending

Detrending can be done in a few different ways.

Two popular ways of doing it are by using **moving averages** and **locally estimated scatterplot smoothing (LOESS) regression**.

Moving averages

Moving average along the time series can be seen as a window that is moved along the time series in steps, and at each step, the average of all the values in the window is recorded.

It helps us **estimate** the **slow change** in a time series, which is the trend. The downside is that the technique is quite **noisy**. Even after smoothing out a time series using this technique, the extracted trend will not be smooth; it will be noisy. The noise should ideally reside with the residuals and not the trend.

Detrending

LOESS

The LOESS (*locally weighted polynomial regression*) algorithm is a non-parametric method that is used to **fit a smooth curve onto a noisy signal**.

We use an **ordinal variable** that moves between the time series as the **independent** variable and the **time series** signal as the **dependent** variable.

For each value in the ordinal variable, the algorithm uses a **fraction of the closest points** and **estimates a smoothed trend** using only those points in a **weighted regression**. The weights in the weighted regression are the closest points to the point in question.

This is given the highest weight and it decays as we move farther away from it. This gives us a very effective tool for modeling the smooth changes in the time series (trend).

Deseasonalizing

The seasonality component can also be estimated in a few different ways.

The two most popular ways of doing this are by using **period-adjusted averages** or a **Fourier series**.

Period adjusted averages

Is a pretty simple technique wherein we calculate a seasonality index for each period in the expected cycle by taking the average values of all such periods over all the cycles.

Additive seasonality: subtract the overall average.

Multiplicative seasonality: divide by the overall average.

The result is a seasonal index that shows how much each month is typically above or below the average.

Deseasonalizing

Month	Year 1	Year 2	Year 3
January	100	110	120
February	90	95	100
March	130	135	140

The period-adjusted average for January is:

$$(100 + 110 + 120) / 3 = 110$$

The period-adjusted average for February is:

$$(90 + 95 + 100) / 3 = 95$$

The period-adjusted average for March is:

$$(130 + 135 + 140) / 3 = 135$$

Additive seasonality

Use an additive index when the seasonal effect is approximately a fixed amount.
The model is:

$$y_t = \text{trend}_t + \text{season}_t + \text{noise}_t$$

For an additive seasonal index, we subtract the average of all period averages from each period average:

$$\text{seasonal index}_j = \text{period average}_j - \text{average of all period averages}$$

For example, if the average of all period averages is 100 and the January average is 110, then the January additive seasonal index is:

$$110 - 100 = 10$$

This means January is typically 10 units above the overall average.

Multiplicative seasonality

Use a multiplicative index when the seasonal effect is proportional to the level of the series. The model is:

$$y_t = \text{trend}_t \times \text{season}_t \times \text{noise}_t$$

For a multiplicative seasonal index, we divide each period average by the average of all period averages:

$$\text{seasonal index}_j = \text{period average}_j / \text{average of all period averages}$$

For example, if the average of all period averages is 100 and the January average is 110, then the January multiplicative seasonal index is:

$$110 / 100 = 1.10$$

This means January is typically 10% above the overall average.

seasonal_decompose from statsmodel

seasonal_decompose from statsmodels.tsa.seasonal is used to split a time series into:

- **Trend**
- **Seasonality**
- **Residual/noise**

It estimates the trend using **moving averages** and the seasonal pattern using **period-based averages**.

Main parameters:

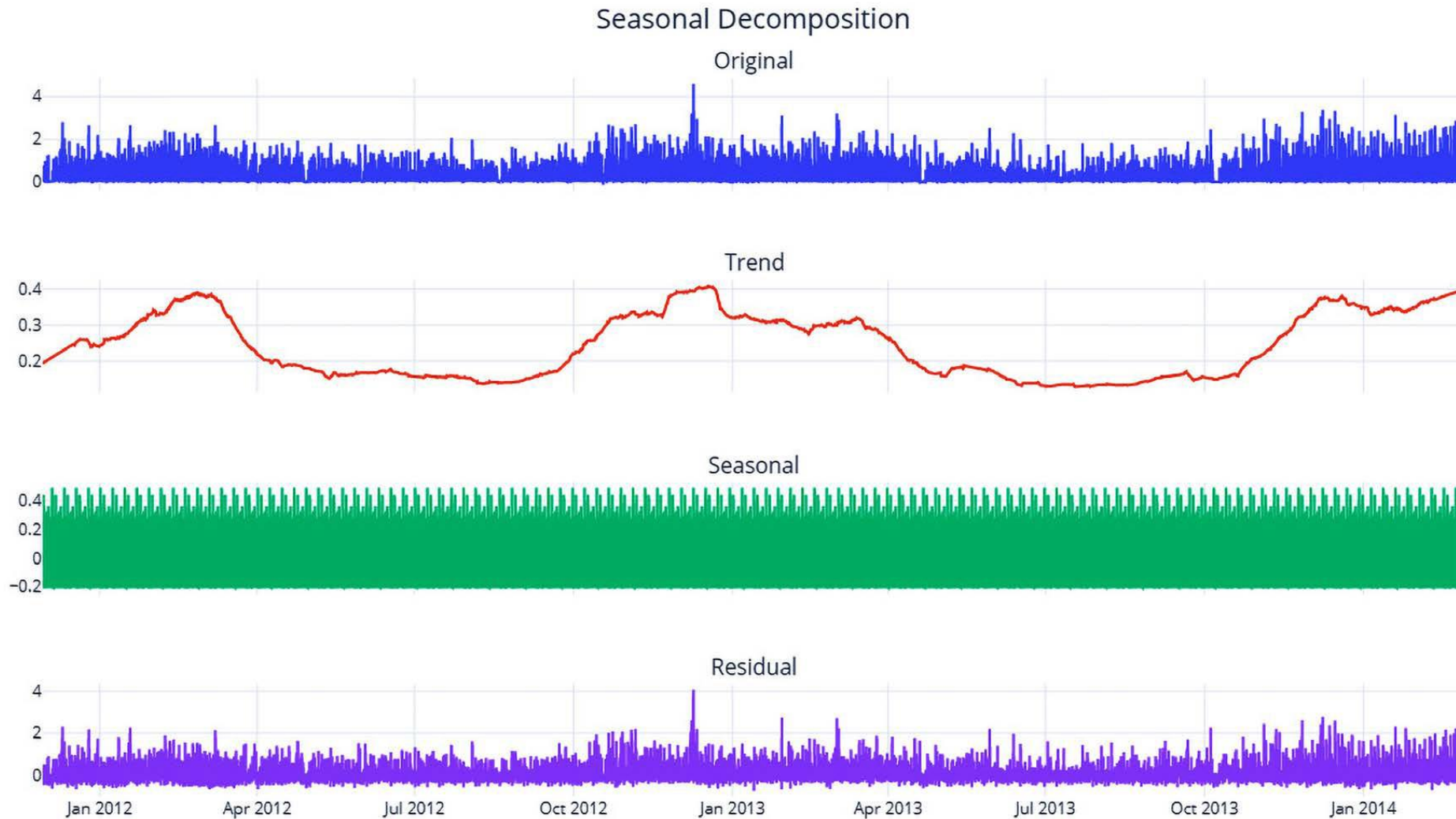
- **period**: length of the seasonal cycle.
- **model**: "additive" or "multiplicative".
- **filt**: controls the moving-average window.
- **extrapolate_trend**: fills trend values at the edges.
- **two_sided**: if True, uses past and future values; if False, uses only past values.

```
#Does not support missing values, so using imputed ts instead  
res = seasonal_decompose(ts, period=7*48, model="additive", extrapolate_  
trend="freq")
```

```
filt=np.repeat(1/(30*48), 30*48)
```

Important note!! seasonal_decompose does **not support missing values**, so missing data should be handled first.

seasonal_decompose from statsmodel



Seasonality and trend decomposition using LOESS (STL)

STL decomposition is used to **split a time series into trend, seasonality, and residuals**.

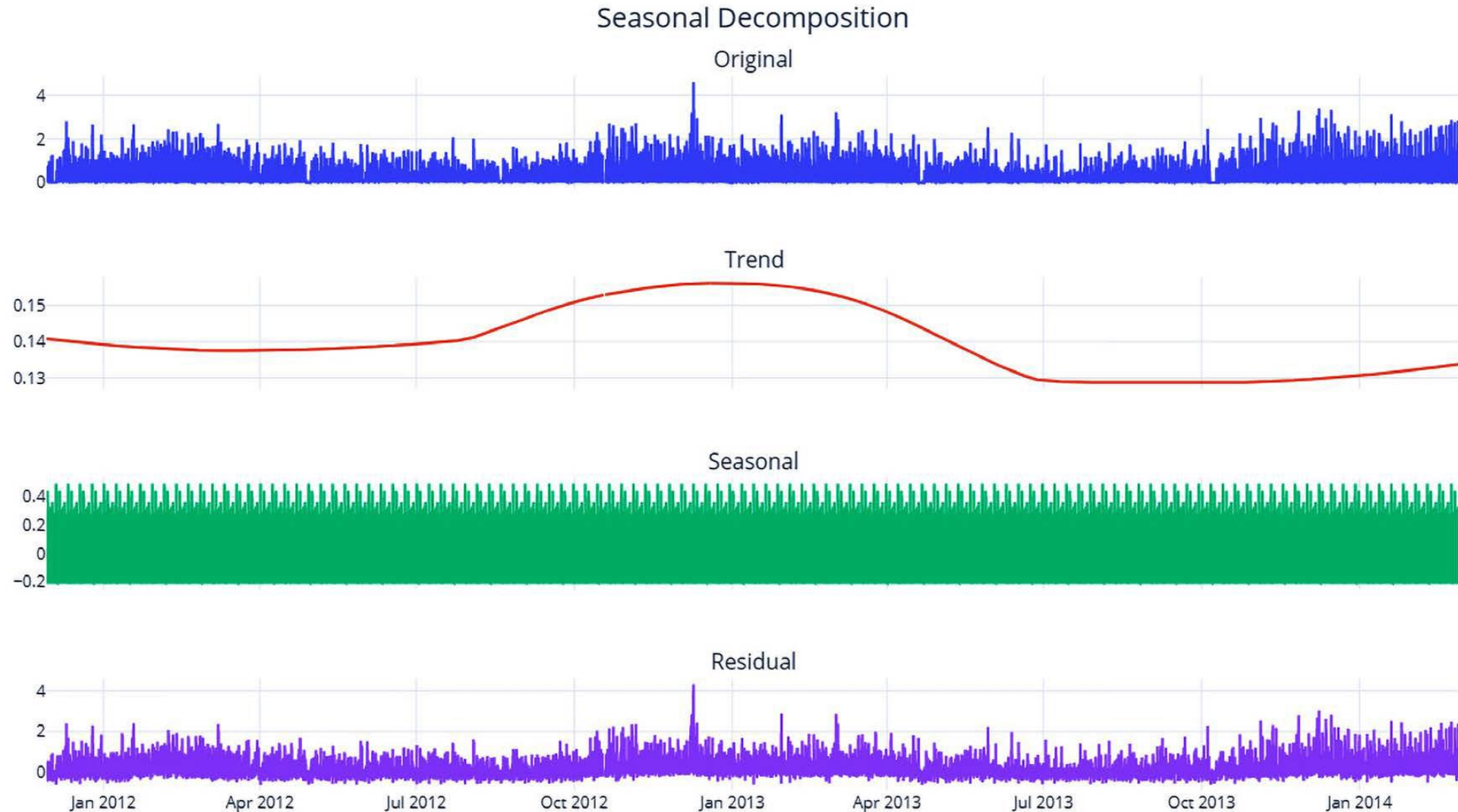
Unlike `seasonal_decompose`, STL uses LOESS to estimate the trend, making it **more flexible for complex trends**. Seasonality is estimated using period-based averages.

```
stl = STL(seasonality_period=7*48, model = "additive")
res_new = stl.fit(ts_df.energy_consumption)
```

Main parameters:

- **seasonality_period**: length of the repeating seasonal cycle.
- **model**: additive or multiplicative decomposition.
- **lo_frac**: fraction of data used in LOESS trend fitting (**default 0.6**).
- **lo_delta**: speeds up computation by using interpolation instead of full regression (**default 0.01**).

Seasonality and trend decomposition using LOESS (STL)



Seasonality and trend decomposition using LOESS (STL)

Let's also look at the decomposition for just one month to see the extracted seasonality patterns clearer:

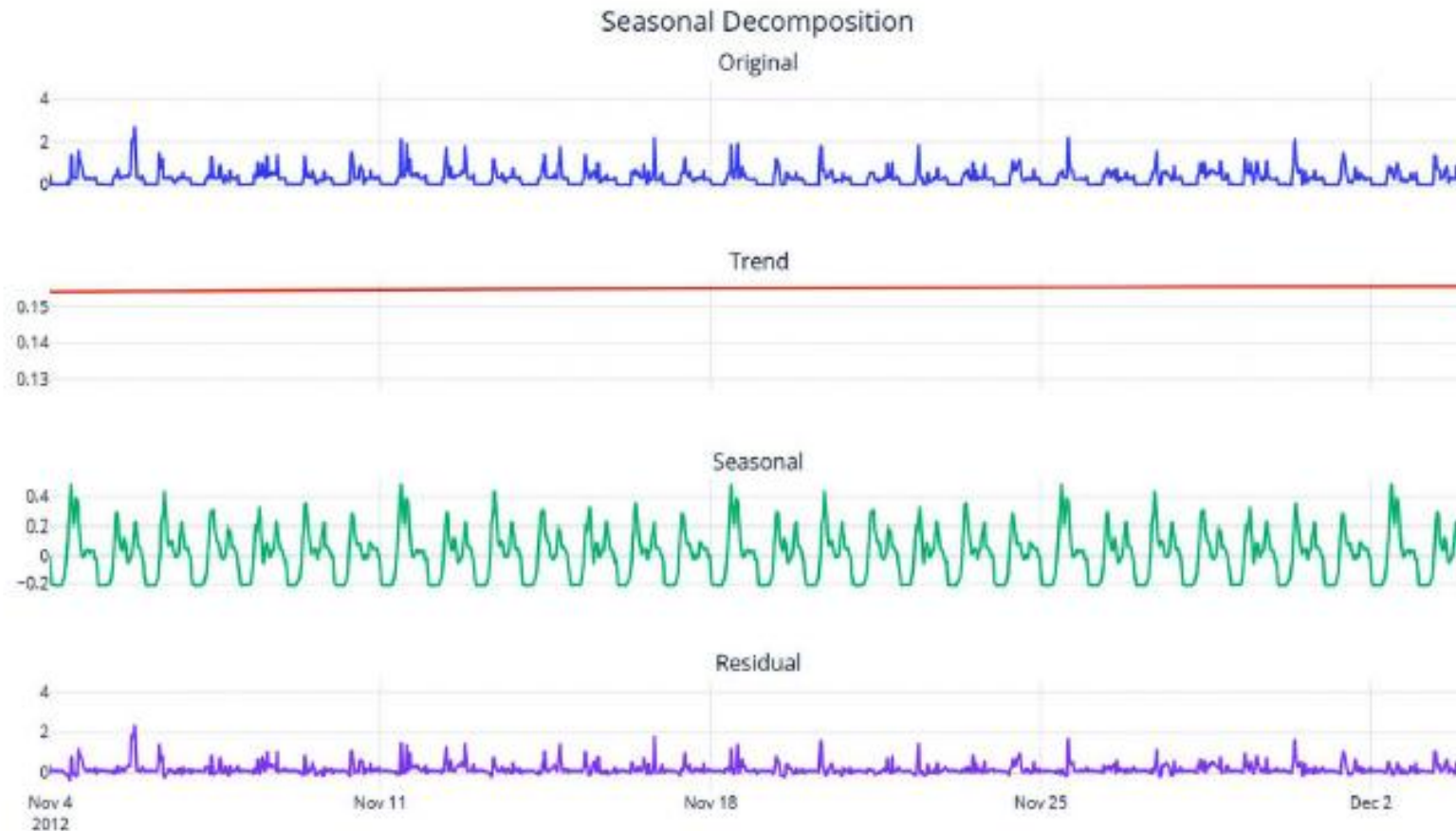


Figure 3.15: STL decomposition (zoomed-in for a month)

Fourier series

A **Fourier series** represents a periodic pattern as a combination of sine and cosine waves. In time series, this is useful because **seasonality is periodic**.

Therefore, we can model seasonal behavior using a small number of Fourier terms.

The cycle length is denoted by **P**.

For example, in monthly data with yearly seasonality, **P = 12**.

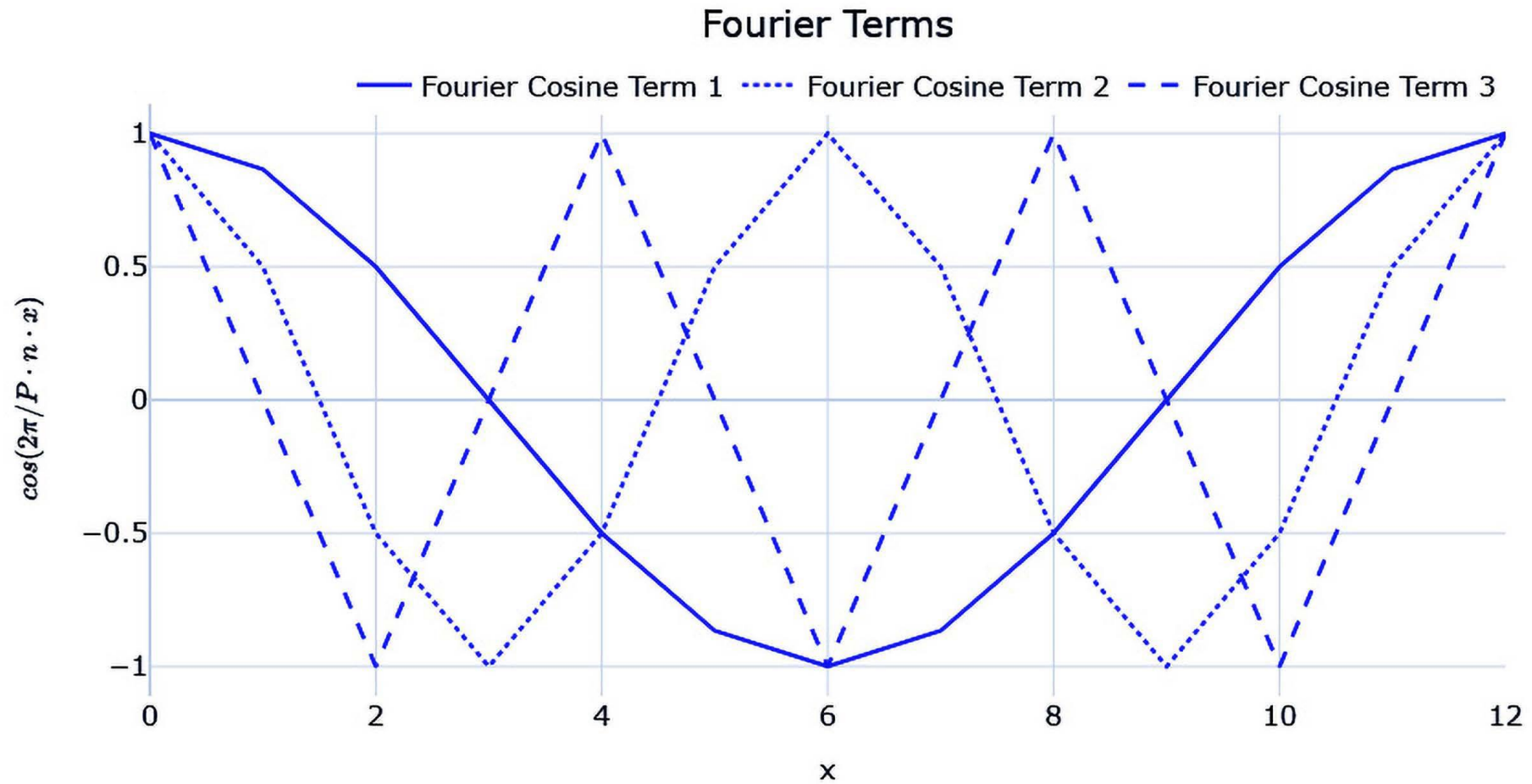
The number of Fourier terms, **N**, controls how complex the seasonal pattern can be:
Small **N** → simple, smooth seasonality.

Large **N** → more complex seasonality, but higher risk of overfitting.

So, Fourier terms help us capture seasonal patterns in a flexible but controlled way.

$$S_{N(x)} = \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cdot \cos\left(\frac{2\pi}{P} \cdot n \cdot x\right) + b_n \cdot \sin\left(\frac{2\pi}{P} \cdot n \cdot x\right) \right)$$

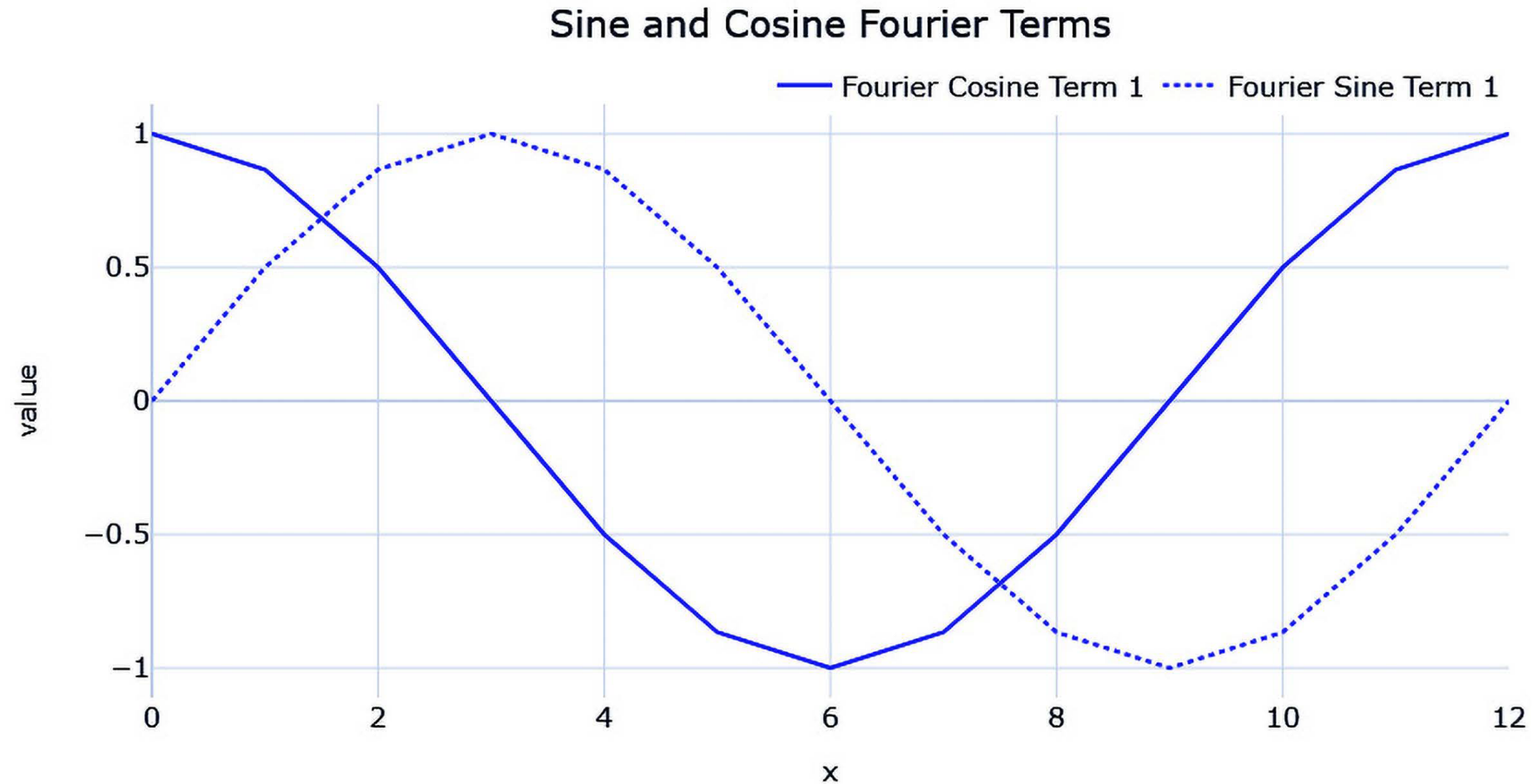
Fourier series



Cosine Fourier terms ($n = 1, 2, 3$)

Fourier series

The sine and cosine waves are complementary to each other



Fourier decomposition

```
stl = FourierDecomposition(seasonality_period="hour", model = "additive", n_
fourier_terms=5)
res_new = stl.fit(pd.Series(ts.squeeze(), index=ts_df.index))
```

We can find the Python implementation for decomposing a time series using Fourier terms in src.

Alternatively, we can create any custom seasonality array that's the same length as the time series that has an ordinal representation of the seasonality. If it is an annual seasonality of daily data, the array would have a minimum value of 1 and a maximum value of 365 as it increases by one every day of the year.

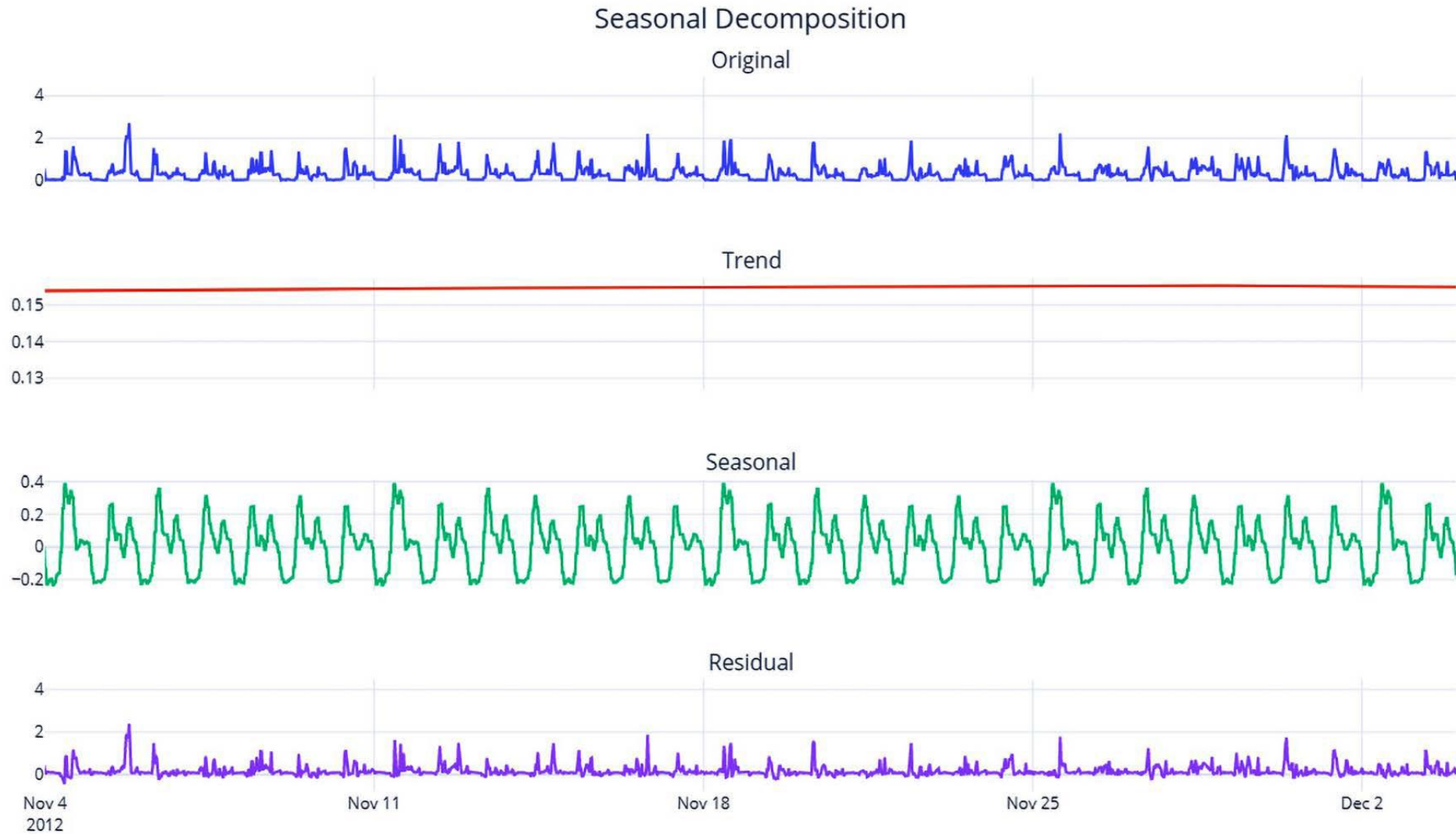
```
#Making a custom seasonality term
ts_df["dayofweek"] = ts_df.index.dayofweek
ts_df["hour"] = ts_df.index.hour
#Creating a sorted unique combination df
map_df = ts_df[["dayofweek", "hour"]].drop_duplicates().sort_
values(["dayofweek", "hour"])
# Assigning an ordinal variable to capture the order
map_df["map"] = np.arange(1, len(map_df)+1)
# mapping the ordinal mapping back to the original df and getting the
seasonality array
seasonality = ts_df.merge(map_df, on=["dayofweek", "hour"], how='left',
validate="many_to_one")['map']
stl = FourierDecomposition(model = "additive", n_fourier_terms=50)
res_new = stl.fit(pd.Series(ts, index=ts_df.index), seasonality=seasonality)
```

Fourier decomposition

The key parameters that are involved in this process are as follows:

- **seasonality_period** is the seasonality to be extracted from the *datetime index*. pandas datetime properties, such as `week_of_day` and `month`, can be used to specify the most prominent seasonality. If left set to `None`, you need to provide the seasonality array while calling `fit`.
- **model** takes `additive` or `multiplicative` as an argument to determine the type of decomposition.
- **n_fourier_terms** determines the number of Fourier terms to be used to extract the seasonality. The more we increase this parameter, the more complex the seasonality that is extracted from the data.
- **lo_frac** is the fraction of the data that will be used to fit the LOESS regression.
- **lo_delta** is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero `lo_delta` significantly decreases computation time.

Fourier decomposition



Multiple seasonality decomposition using LOESS (MSTL)

High-frequency time series, such as hourly or daily data, may contain **multiple seasonal patterns** at the same time, for example hourly, weekly, and yearly seasonality.

MSTL extends STL decomposition to handle several seasonalities instead of only one. This gives a better decomposition because remaining seasonal patterns are not incorrectly left inside the residuals.

Main parameters:

- **seasonality_periods**: list of seasonal cycles to extract.
- **seasonality_model**: uses either period averages or Fourier terms.
- **model**: additive or multiplicative decomposition.
- **n_fourier_terms**: controls the complexity of Fourier-based seasonality.
- **lo_frac**: fraction of data used for LOESS trend fitting.
- **lo_delta**: speeds up LOESS using interpolation.

```
stl = MultiSeasonalDecomposition(seasonal_model="fourier", seasonality_
periods=["day_of_year", "day_of_week", "hour"], model = "additive", n_fourier_
terms=10)
res_new = stl.fit(pd.Series(ts, index=ts_df.index))
```

Multiple seasonality decomposition using LOESS (MSTL)

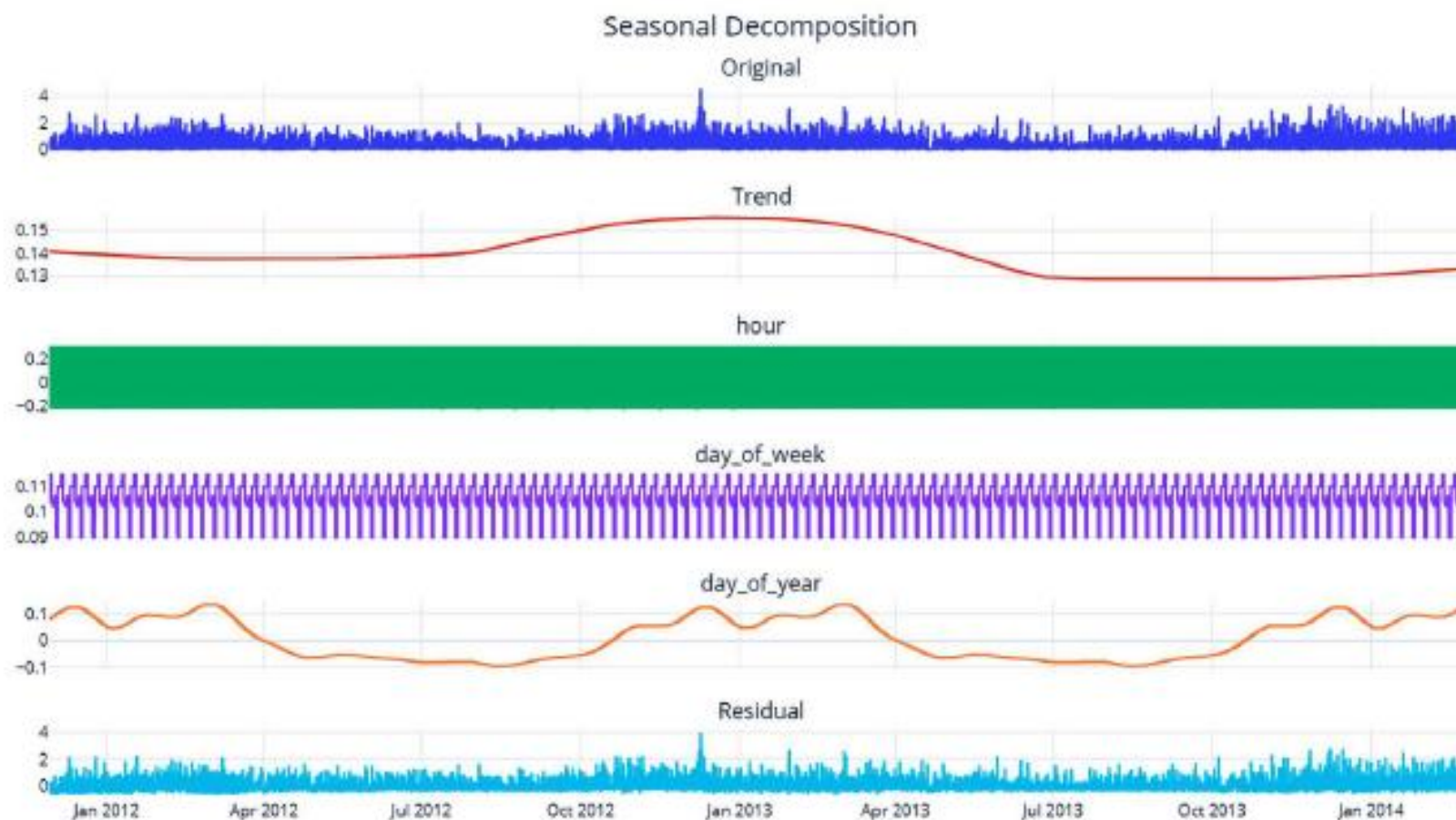


Figure 3.17: Multiple seasonality decomposition using Fourier terms

Multiple seasonality decomposition using LOESS (MSTL)

Here, we can see that the `day_of_week` seasonality has been extracted. To see the `day_of_week` and hour seasonal components, we need to zoom in a bit:

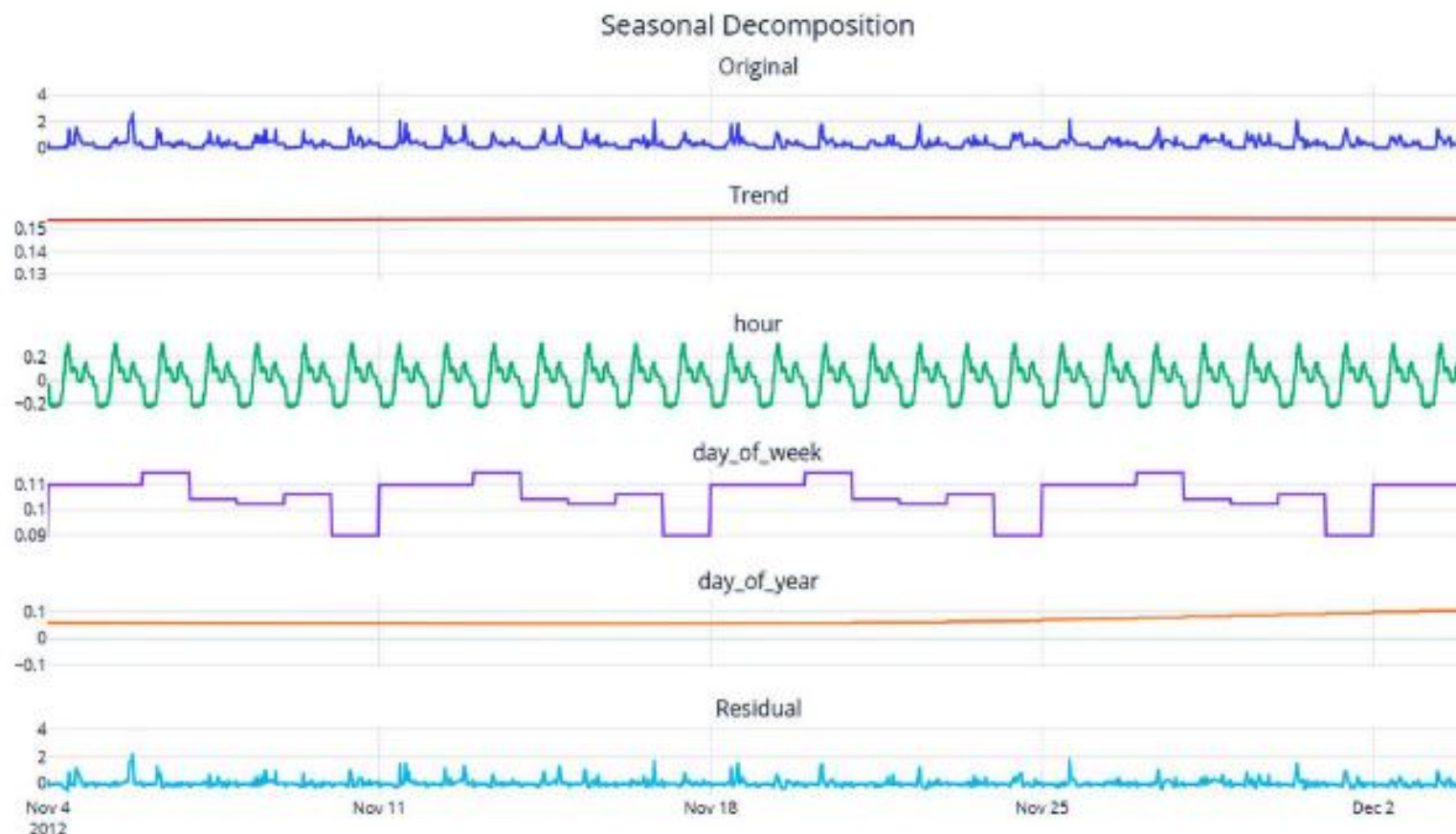


Figure 3.18: Multiple seasonality decomposition using Fourier terms (zoomed-in for a month)

The four techniques

Implementation	Trend	Seasonal	Supports Multiple Seasonality?	Supports Missing?
Seasonality decomposition	Moving Averages	Period-Adjusted Averages	No	No
STL	LOESS	Period-Adjusted Averages	No	Yes
Fourier decomposition	LOESS	Fourier Terms	No	No
Multiple seasonality decomposition	LOESS	Period-Adjusted Averages / Fourier Terms	Yes	No

Table 3.1: Different seasonal decomposition techniques

Detecting and treating outliers

An **outlier** is an unusual data point that differs strongly from the rest of the time series.

It may result from **errors**, **faulty measurements**, or **rare events**.

Detecting and treating outliers helps clean the data and can improve forecasting model performance.



Standard Deviation

The **standard deviation rule** marks values as outliers if they are far from the mean, usually beyond:

$$\mu \pm 3\sigma$$

Under a normal distribution, about 99% of values should fall within this range, so values outside it are considered rare.

The threshold can be adjusted: larger multiples detect fewer outliers.

For seasonal time series, apply this rule to the **residuals after removing seasonality**, not to the raw data. This method relies on the assumption that the data is **approximately normally distributed**.

IQR

The **IQR method** detects outliers using **quartiles** instead of **mean** and **standard deviation**.

$$IQR = Q3 - Q1$$

Outliers are values outside:

$$Q1 - 1.5 \cdot IQR$$

and

$$Q3 + 1.5 \cdot IQR$$

This method is more robust than standard deviation because quartiles are less affected by extreme values.

For time series, it is usually better to remove trend/seasonality before applying it.

Isolation Forest

Isolation Forest is an unsupervised anomaly detection method that uses random decision trees to isolate unusual points. **Outliers** are easier to isolate, so they appear in **shorter tree branches**, while normal points require longer paths.

The anomaly score is based on how quickly a point is isolated. In scikit-learn, it is implemented as **sklearn.ensemble.IsolationForest**, with `contamination` controlling the expected proportion of anomalies.

However, it **ignores time order** and only detects values that look unusual.

Extreme studentized deviate (ESD) and seasonal ESD (S-ESD)

ESD is a statistical outlier detection method based on Grubbs's test, **assuming normally distributed data**. It repeatedly identifies and **removes one outlier at a time** while **updating the critical threshold**.

For time series, **S-ESD** extends this idea by first **removing seasonality**, making it useful for detecting anomalies in seasonal data. Unlike simpler methods that require manual threshold tuning, S-ESD only needs an **upper bound on the expected number of outliers** and then identifies them automatically.

Treating outliers

Outlier correction should not be done automatically. **Detected outliers** should first be **verified**, because they **may represent meaningful patterns** rather than errors.

For **small** datasets, inspect them **manually**.

For **many** time series, use **automated** treatment such as replacing outliers with a heuristic value or treating them as missing values and imputing them.

Outlier correction is **optional**, especially with modern ML/DL models, so its impact should be tested experimentally.

Load Data

```
local_path = Path("machine_temperature_system_failure.csv")
remote_url = "https://raw.githubusercontent.com/numenta/NAB/master/data/realKnownCause/machine_tem

if local_path.exists():
    df = pd.read_csv(local_path)
    print("Loaded local file:", local_path)
else:
    df = pd.read_csv(remote_url)
    print("Loaded remote file:", remote_url)

df.head()
```

	timestamp	value
0	2013-12-02 21:15:00	73.967322
1	2013-12-02 21:20:00	74.935882
2	2013-12-02 21:25:00	76.124162
3	2013-12-02 21:30:00	78.140707
4	2013-12-02 21:35:00	79.329836

```
df["timestamp"] = pd.to_datetime(df["timestamp"])
df = df.sort_values("timestamp").set_index("timestamp")

# Remove duplicate timestamps, if any
df = df[~df.index.duplicated(keep="first")]

# Keep only the sensor value
ts_df = df[["value"]].copy()

# Regularize to 5-minute frequency and interpolate occasional missing points
ts_df = ts_df.asfreq("5min")
ts_df["value"] = ts_df["value"].interpolate(method="time")

ts_df.head()
```

timestamp	
2013-12-02 21:15:00	73.967322
2013-12-02 21:20:00	74.935882
2013-12-02 21:25:00	76.124162
2013-12-02 21:30:00	78.140707
2013-12-02 21:35:00	79.329836

EDA

```
print("Start:", ts_df.index.min())
print("End:", ts_df.index.max())
print("Number of observations:", len(ts_df))
print("Missing values:")
display(ts_df.isna().sum())

ts_df.describe()
```

```
fig = px.line(
    ts_df,
    x=ts_df.index,
    y="value",
    title="Machine Temperature Sensor Time Series"
)
fig.update_layout(
    xaxis_title="Timestamp",
    yaxis_title="Temperature Sensor Value",
    template="plotly_white"
)
fig.show()
```

```
Start: 2013-12-02 21:15:00
End: 2014-02-19 15:25:00
Number of observations: 22683
Missing values:
```

0

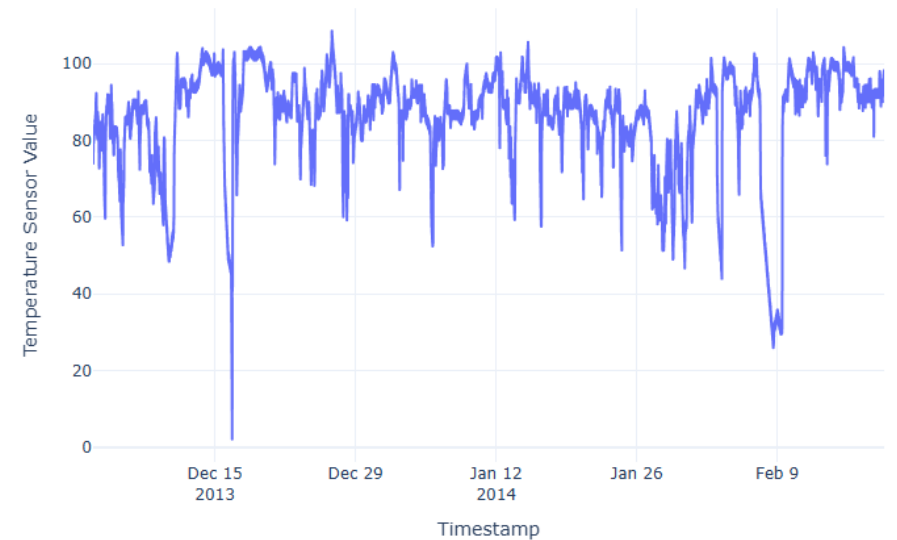
value 0

dtype: int64

	value
count	22683.000000
mean	85.922177
std	13.749252
min	2.084721
25%	83.074740
50%	89.403332
75%	94.012535
max	108.510543



Machine Temperature Sensor Time Series



Decomposing Time Series

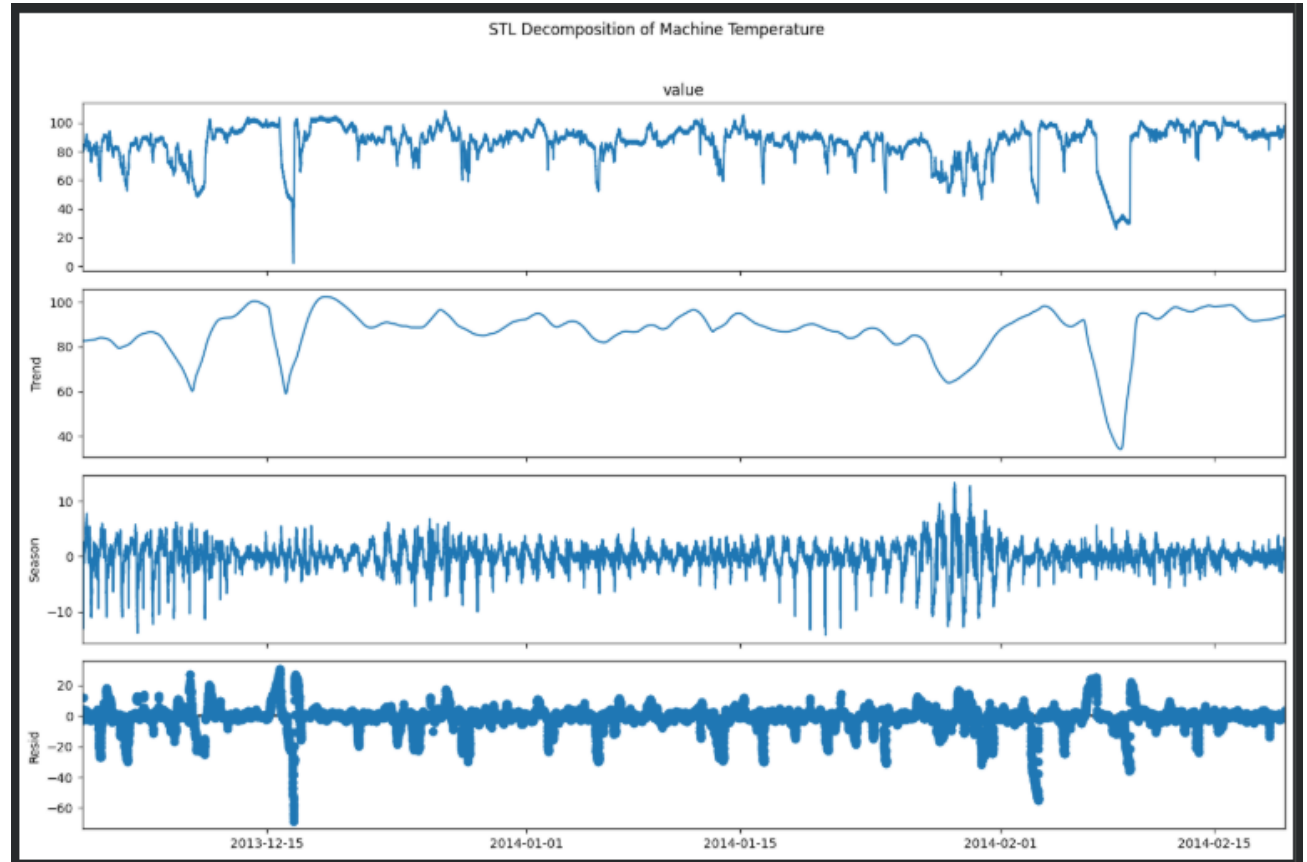
Daily seasonality = $24 \times 12 = 288$ observations

```
series = ts_df["value"]

stl = STL(series, period=288, robust=True)
stl_result = stl.fit()

residuals = stl_result.resid

fig = stl_result.plot()
fig.set_size_inches(14, 9)
plt.suptitle("STL Decomposition of Machine Temperature", y=1.02)
plt.tight_layout()
plt.show()
```



Helper Functions

```
def plot_outliers(index, values, outlier_mask, method, ylabel="Temperature Sensor Value"):
    outlier_mask = np.asarray(outlier_mask).astype(bool)
    values = np.asarray(values)
    index = np.asarray(index)

    fig = go.Figure()
    fig.add_trace(go.Scatter(
        x=index,
        y=values,
        mode="lines",
        name="Original"
    ))
    fig.add_trace(go.Scatter(
        x=index[outlier_mask],
        y=values[outlier_mask],
        mode="markers",
        marker_symbol="star",
        marker_size=7,
        name="Detected outliers"
    ))
    fig.update_layout(
        title=f"{method}: {outlier_mask.sum()} outliers ({outlier_mask.sum() / len(values) * 100:.2f}%)",
        xaxis_title="Timestamp",
        yaxis_title=ylabel,
        template="plotly_white"
    )
    return fig
```

Helper Functions

```
def detect_outlier_sd(values, sd_multiple=3):
    values = pd.Series(values).astype(float)
    z_scores = (values - values.mean()) / values.std()
    return np.abs(z_scores) > sd_multiple

def detect_outlier_iqr(values, iqr_multiple=1.5):
    values = pd.Series(values).astype(float)
    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - iqr_multiple * iqr
    upper_bound = q3 + iqr_multiple * iqr
    return (values < lower_bound) | (values > upper_bound)

def detect_outlier_isolation_forest(values, contamination=0.01, random_state=42):
    values = np.asarray(values).reshape(-1, 1)
    model = IsolationForest(
        contamination=contamination,
        random_state=random_state
    )
    labels = model.fit_predict(values)
    return labels == -1
```

Standard deviation

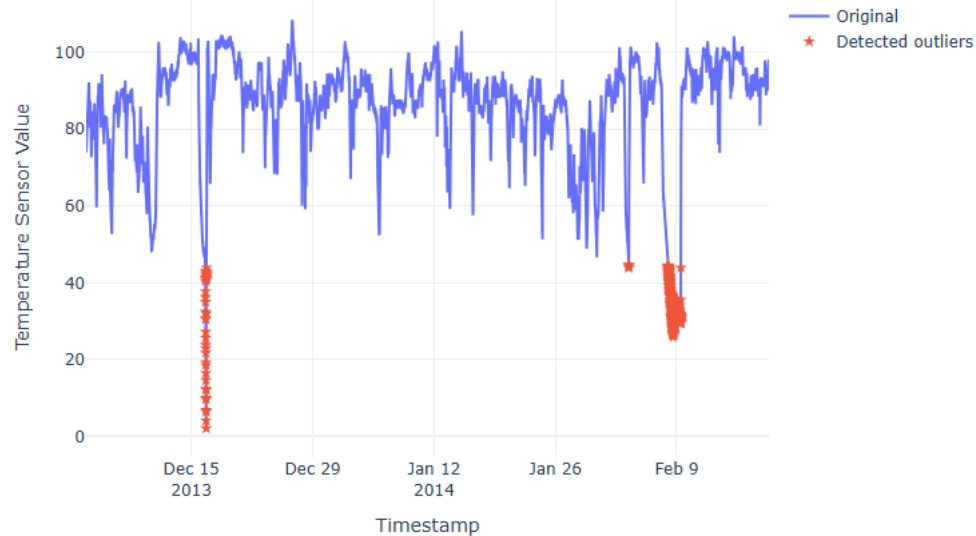
```
results = pd.DataFrame(columns=["# of Outliers", "% of Outliers"])
values = ts_df["value"].values

sd_mask = detect_outlier_sd(values, sd_multiple=3)

results.loc["3 Standard Deviations", "# of Outliers"] = sd_mask.sum()
results.loc["3 Standard Deviations", "% of Outliers"] = sd_mask.sum() / len(values) * 100

fig = plot_outliers(ts_df.index, values, sd_mask, method="Standard Deviation Rule")
fig.show()
```

Standard Deviation Rule: 462 outliers (2.04%)

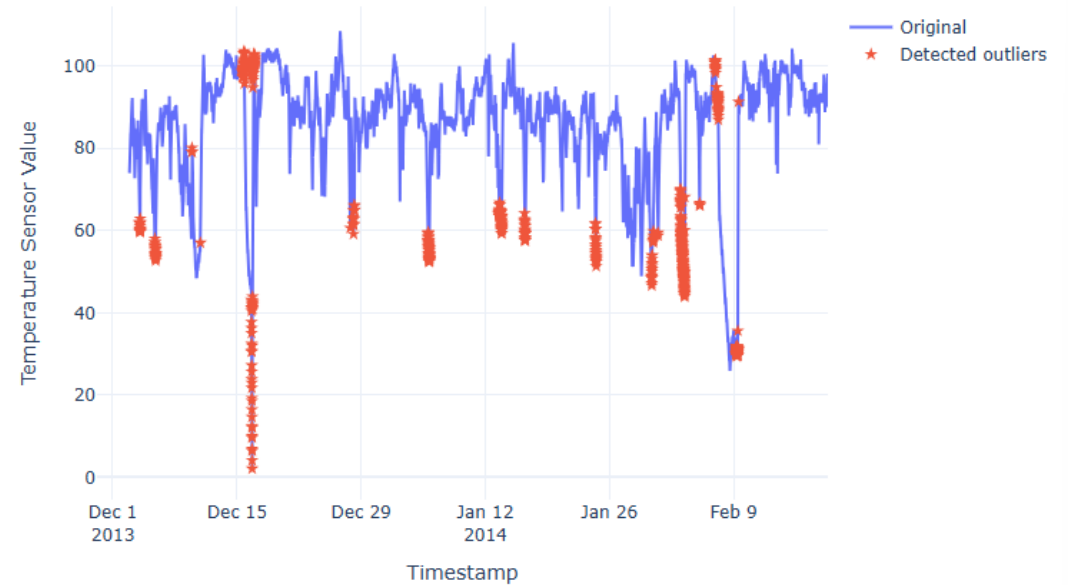


```
resid_mask = detect_outlier_sd(residuals, sd_multiple=3)

results.loc["3 SD on STL Residuals", "# of Outliers"] = resid_mask.sum()
results.loc["3 SD on STL Residuals", "% of Outliers"] = resid_mask.sum() / len(values) * 100

fig = plot_outliers(ts_df.index, values, resid_mask, method="STL Residual Outlier Detection")
fig.show()
```

STL Residual Outlier Detection: 597 outliers (2.63%)



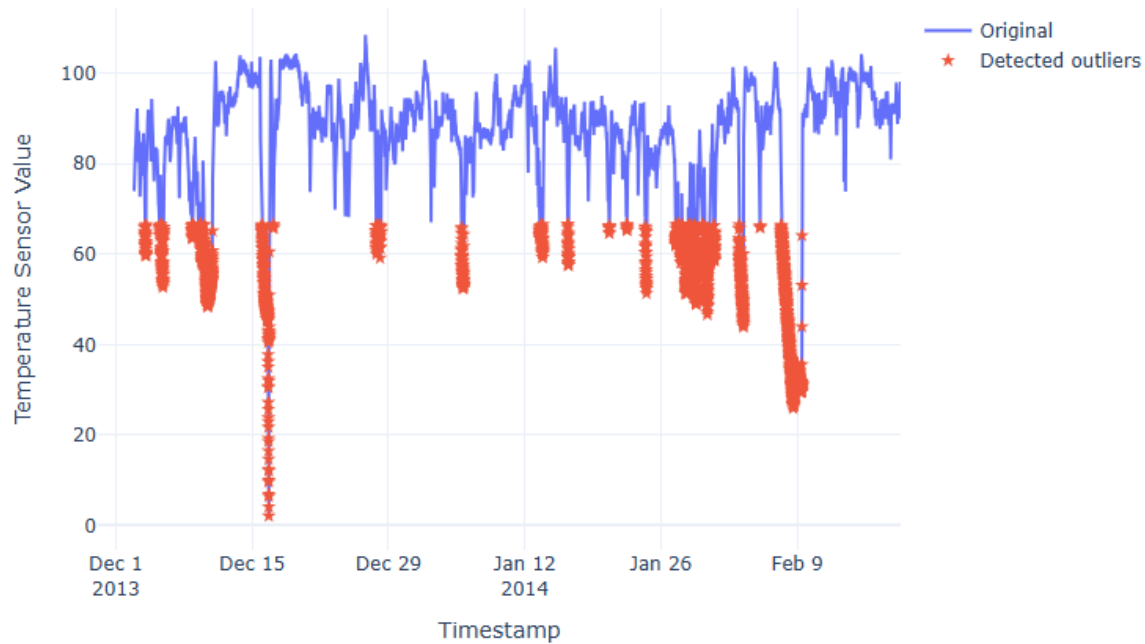
IQR

```
iqr_mask = detect_outlier_iqr(values, iqr_multiple=1.5)

results.loc["1.5 IQR", "# of Outliers"] = iqr_mask.sum()
results.loc["1.5 IQR", "% of Outliers"] = iqr_mask.sum() / len(values) * 100

fig = plot_outliers(ts_df.index, values, iqr_mask, method="IQR Rule")
fig.show()
```

IQR Rule: 2294 outliers (10.11%)

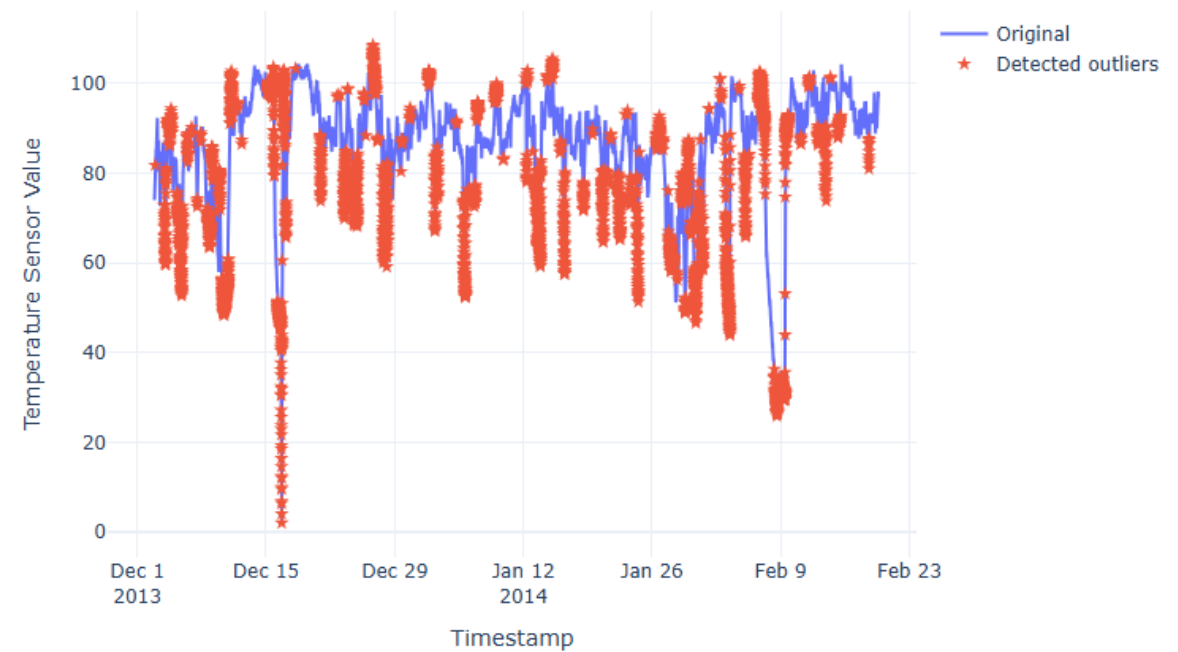


```
iqr_mask = detect_outlier_iqr(residuals, iqr_multiple=1.5)

results.loc["1.5 IQR", "# of Outliers"] = iqr_mask.sum()
results.loc["1.5 IQR", "% of Outliers"] = iqr_mask.sum() / len(values) * 100

fig = plot_outliers(ts_df.index, values, iqr_mask, method="IQR Rule")
fig.show()
```

IQR Rule: 3804 outliers (16.77%)



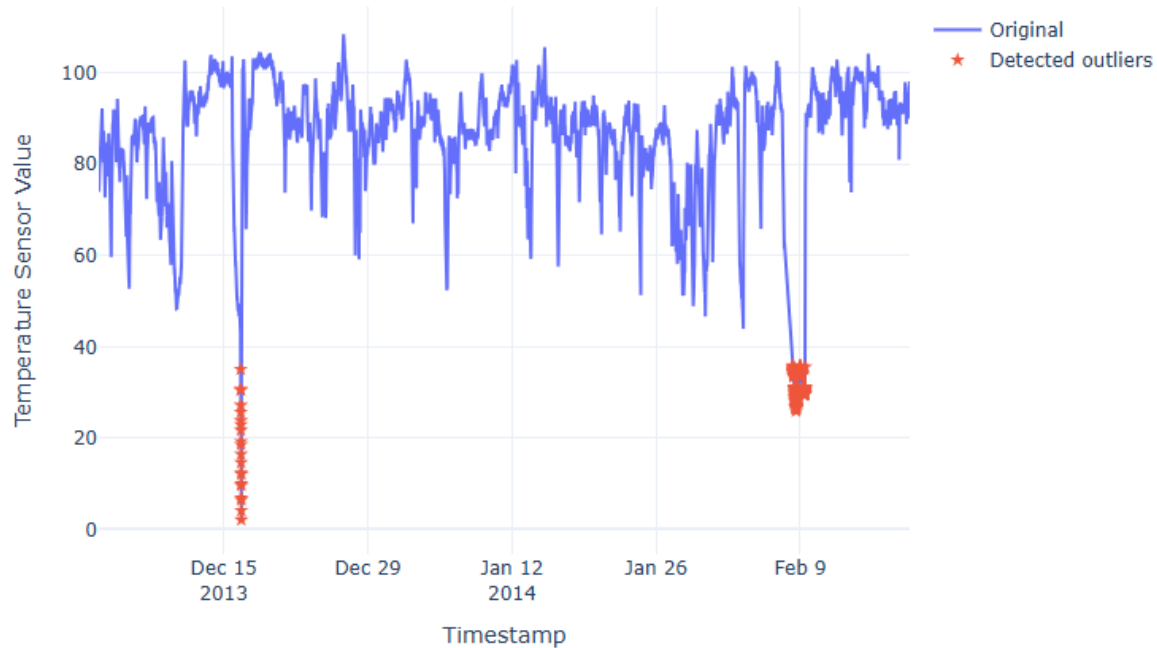
Isolation Forest

```
iforest_mask = detect_outlier_isolation_forest(values, contamination=0.01)

results.loc["Isolation Forest", "# of Outliers"] = iforest_mask.sum()
results.loc["Isolation Forest", "% of Outliers"] = iforest_mask.sum() / len(values) * 100

fig = plot_outliers(ts_df.index, values, iforest_mask, method="Isolation Forest")
fig.show()
```

Isolation Forest: 215 outliers (0.95%)

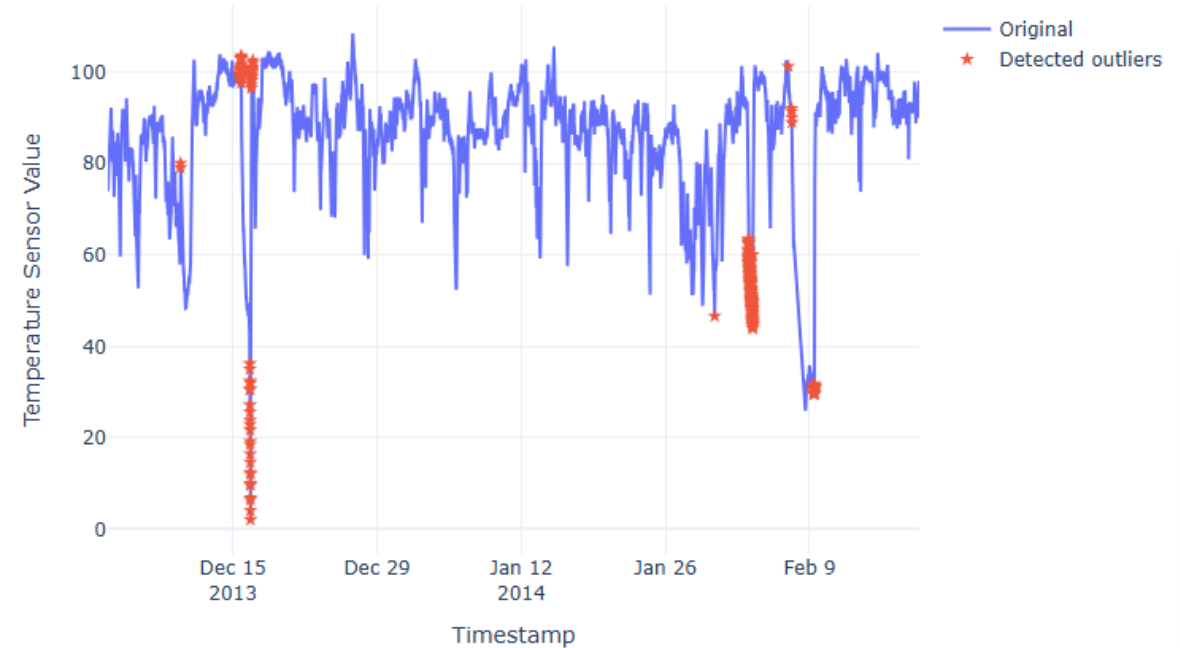


```
iforest_mask = detect_outlier_isolation_forest(residuals, contamination=0.01)

results.loc["Isolation Forest", "# of Outliers"] = iforest_mask.sum()
results.loc["Isolation Forest", "% of Outliers"] = iforest_mask.sum() / len(values) * 100

fig = plot_outliers(ts_df.index, values, iforest_mask, method="Isolation Forest")
fig.show()
```

Isolation Forest: 220 outliers (0.97%)



Seasonal ESD (S-ESD)

```
import sys
import numpy as np
import pandas as pd

# 1. Correct package installation and import
!{sys.executable} -m pip install sesd

import sesd

def detect_outlier_sesd(series, period, max_anomalies_fraction=0.01, alpha=0.05):
    """Detects outliers using the official Seasonal ESD library.

    Args:
        series (pd.Series or np.ndarray): The time series data.
        period (int): The periodicity of the time series.
        max_anomalies_fraction (float): Max percentage of anomalies allowed.
        alpha (float): The significance level for the test.

    Returns:
        np.ndarray: A boolean mask where True indicates an outlier.
    """
    # Convert input to a clean 1D numpy array (handles series or residuals safely)
    data_array = np.asarray(series)
    n_samples = len(data_array)

    # Calculate absolute maximum number of anomalies (must be at least 1)
    max_anomalies = max(1, int(n_samples * max_anomalies_fraction))

    # The package function is seasonal_esd, which returns indices of outliers
    outlier_indices = sesd.seasonal_esd(
        data_array, periodicity=period, max_anomalies=max_anomalies, alpha=alpha
    )

    # Initialize a boolean mask of False values
    mask = np.zeros(n_samples, dtype=bool)

    # Flip the detected positions to True
    if len(outlier_indices) > 0:
        mask[outlier_indices] = True

    return mask
```

```
# 1. Apply S-ESD to the original series
sesd_original_mask = detect_outlier_sesd(
    values, period=period_daily, max_anomalies_fraction=0.01
)

results.loc["S-ESD (Original)", "# of Outliers"] = sesd_original_mask.sum()
results.loc["S-ESD (Original)", "% of Outliers"] = (
    sesd_original_mask.sum() / len(values) * 100
)

fig1 = plot_outliers(
    ts_df.index, values, sesd_original_mask, method="S-ESD (Original)"
)
fig1.show()

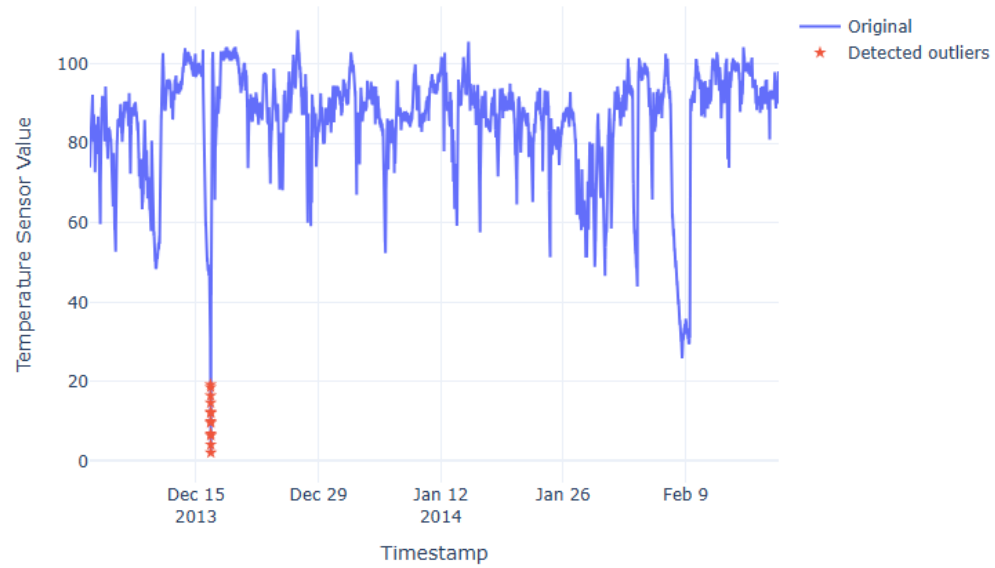
# 2. Apply S-ESD to the STL residuals
sesd_resid_mask = detect_outlier_sesd(
    residuals, period=period_daily, max_anomalies_fraction=0.01
)

results.loc["S-ESD (Residuals)", "# of Outliers"] = sesd_resid_mask.sum()
results.loc["S-ESD (Residuals)", "% of Outliers"] = (
    sesd_resid_mask.sum() / len(values) * 100
)

fig2 = plot_outliers(
    ts_df.index, values, sesd_resid_mask, method="S-ESD (Residuals)"
)
fig2.show()
```

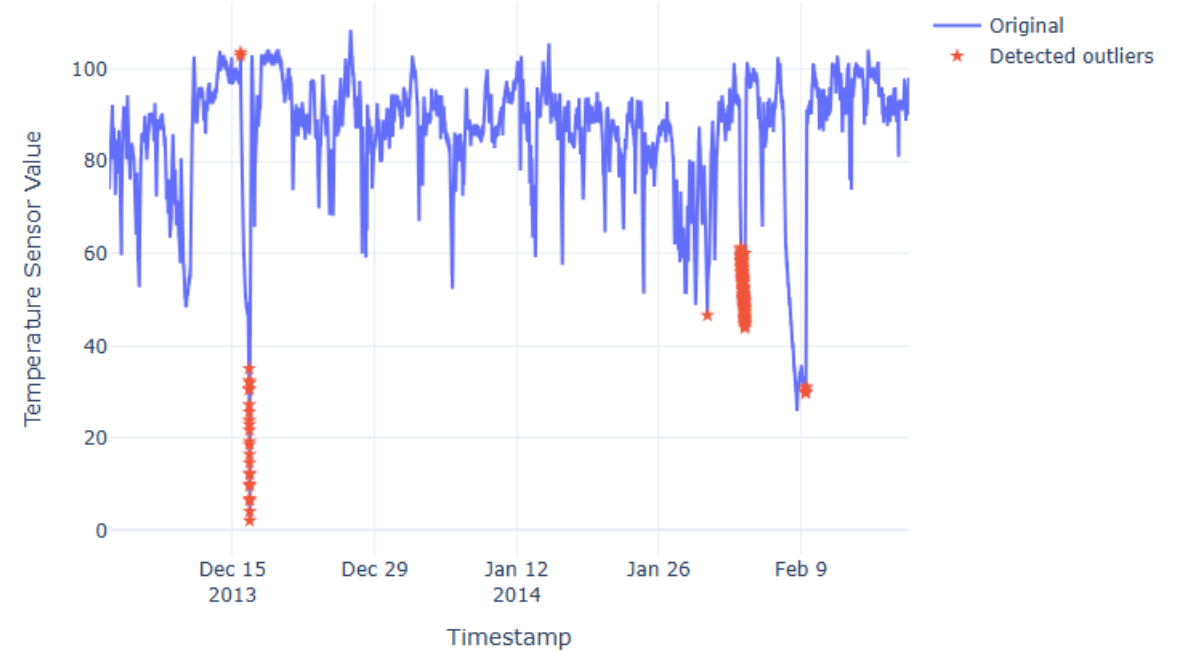
Seasonal ESD (S-ESD)

S-ESD (Original): 12 outliers (0.05%)



S-ESD (Residuals): 150 outliers (0.66%)

S-ESD (Residuals): 150 outliers (0.66%)



Outliers detection using different techniques

```
results.style.format({"% of Outliers": "{:.2f}%"})
```

	# of Outliers	% of Outliers
3 std (Originals)	462	2.04%
3 std (Residuals)	597	2.63%
1.5 IQR (Originals)	2294	10.11%
1.5 IQR (Residuals)	3804	16.77%
Isolation Forest (Originals)	215	0.95%
Isolation Forest (Residuals)	220	0.97%
S-ESD (Original)	12	0.05%
S-ESD (Residuals)	150	0.66%

Treating Outliers

```
treated_df = ts_df.copy()
treated_df["is_outlier"] = iforest_mask
treated_df["value_treated"] = treated_df["value"].mask(treated_df["is_outlier"])
treated_df["value_treated"] = treated_df["value_treated"].interpolate(method="time")
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=treated_df.index,
    y=treated_df["value"],
    mode="lines",
    name="Original"
))
fig.add_trace(go.Scatter(
    x=treated_df.index,
    y=treated_df["value_treated"],
    mode="lines",
    name="Treated"
))
fig.update_layout(
    title="Original vs. Treated Machine Temperature Series",
    xaxis_title="Timestamp",
    yaxis_title="Temperature Sensor Value",
    template="plotly_white"
)
fig.show()
```

Original vs. Treated Machine Temperature Series

